理解 MHA、MQA、GQA 与 MLA

Understanding MHA, MQA, GQA & MLA

原文作者：@techNmak。

先列专业术语对照表，再提供逐页完整中文译文。每个原文页面的译文之后附原文矢量对照页，完整保留公式、图表、代码和英文引用，便于核对。

正文中文：LXGW WenKai（霞鹜文楷）；英文与数字：Times New Roman。原文对照页保留原始字体。

本文采用翻译辅助服务生成初稿，统一技术术语并检查结构、字体和导航；涉及精确数值、公式或实现细节时，可直接对照相邻原文页。

专业术语翻译对照表

英文原词	中文译名	缩写／符号
Multi-head Latent Attention	多头潜在注意力	MLA
Grouped-query Attention	分组查询注意力	GQA
Multi-head Attention	多头注意力	MHA
Model Capacity	模型容量	
Inference	推理	
KV Cache	键值缓存	KV Cache
Query	查询	Q
Value	值	V
Key	键	K
Latent Representation	潜在表示	
Key/Value Heads	键值头	KV Head
Representation	表示	
Cache Capacity	缓存容量	
Cache Payload	缓存有效载荷	
Mental Model	理解框架	
Query Heads	查询头	
Attention	注意力机制	Attention
Training	训练	
Decoder	解码器	
Values	值	
Token	词元	
Heads	头	
Model	模型	
Keys	键	
Causal Self-attention	因果自注意力	
Attention Head	注意力头	
Tokens	词元	
Attention Heads	注意力头	
Transformer	Transformer 模型	Transformer
Query Head	查询头	
Queries	查询	
Vectors	向量	

英文原词	中文译名	缩写／符号
Batch Size	批大小	
Head Width	头宽度	
Precision	精度	
Incremental Decoding	增量解码	
Activation	激活	
Serving	推理服务	
Value Heads	值头	
Key Heads	键头	
Tensor	张量	
Batching	批处理	
KV Heads	键值头	
Kernels	内核	
Tiling	分块	
Attention Distribution	注意力分布	
Multi-query Attention	多查询注意力	MQA
Output Projection	输出投影	
Dense Attention	稠密注意力	
Baseline	基线	
Key/Value Head	键值头	KV Head
Pretraining	预训练	
Pooling	池化	
Decoder-only LLM	仅解码器大语言模型	
Benchmark	基准测试	
Kernel	内核	
Models	模型	
Fragmentation	碎片化	
Activations	激活	
Memory Traffic	内存流量	
Throughput	吞吐量	
Evidence	证据	
Projection Absorption	投影吸收	
Hidden State	隐藏状态	
Vector	向量	
Head Dimension	头维度	

英文原词	中文译名	缩写／符号
Matrix Multiplication	矩阵乘法	
Projection Matrices	投影矩阵	
Latent Compression	潜在压缩	
Compressed Latent	压缩潜在表示	
Up-projection	升维投影	
Linear Projections	线性投影	
Operator	算子	
Matrix	矩阵	
Positional Encoding	位置编码	
Decoupled RoPE	解耦 RoPE	
Residual Stream	残差流	
FlashAttention	FlashAttention（方法名保留）	
Softmax	Softmax 归一化	Softmax
Combine	合并	
PagedAttention	PagedAttention（方法名保留）	
Quantization	量化	
Tensor Shape	张量形状	
Fused Kernel	融合内核	
Mask	掩码	
KV-head Multiplicity	键值头数量	
Allocator Overhead	分配器开销	
Memory Footprint	内存占用	
Quantized	量化后的	
Latency	延迟	
FLOPs	浮点运算次数	FLOPs
Rotary Position Embedding	旋转位置嵌入	RoPE
Large Language Model	大语言模型	LLM
Mixture-of-Experts	混合专家	MoE
Exact Attention	精确注意力	
Head Dimensions	头维度	

目录导航

点击章节标题跳转到中文译文，点击“原文”核对公式和图表。页码按本译本最终分页生成。

封面与说明 · 第 1 页

专业术语翻译对照表 · 第 2 页

了解 MHA、MQA、GQA 和 MLA · 第 7 页 ［原文 1］

如何阅读本手册 · 第 9 页 ［原文 2］

1. 从一个 注意力头（Attention Head） 开始 · 第 11 页 ［原文 3］

2、为什么存在多个头（Heads） · 第 13 页 ［原文 4］

3.多头注意力（Multi-head Attention）（MHA） · 第 15 页 ［原文 5］

4. 训练（Training） 和解码强调不同的资源 · 第 17 页 ［原文 6］

5. KV-cache公式，推导 · 第 19 页 ［原文 7］

6. 解码带宽：为什么 键值头（KV Heads） 帮助较少 · 第 21 页 ［原文 8］

7.多查询注意力（Multi-query Attention）（MQA） · 第 23 页 ［原文 9］

8. MQA 能节省什么，不能节省什么 · 第 25 页 ［原文 10］

9.分组查询注意力（Grouped-query Attention）（GQA） · 第 27 页 ［原文 11］

10. MHA、GQA 和 MQA 一张图 · 第 29 页 ［原文 12］

11. GQA 也是一个向上训练的良方 · 第 31 页 ［原文 13］

12. 原始 GQA 结果实际显示了什么 · 第 33 页 ［原文 14］

13. 32Q / 8KV 分组示例 · 第 35 页 ［原文 15］

14. 工作缓存计算：MHA vs GQA vs MQA · 第 37 页 ［原文 16］

15. 磁头共享的变化大于内存容量的变化 · 第 39 页 ［原文 17］

16. 现代模型（Models）使GQA成为主流 · 第 41 页 ［原文 18］

17. MLA 是设计空间的一个不同分支 · 第 43 页 ［原文 19］

18.低秩联合K/V压缩 · 第 45 页 ［原文 20］

19.吸收技巧：不要仅仅为了缓存而重建完整的K/V · 第 47 页 ［原文 21］

20. 查询（Query） 压缩是一个单独的想法 · 第 49 页 ［原文 22］

21. 为什么普通RoPE会破坏简单的MLA吸收 · 第 51 页 ［原文 23］

22.解耦 RoPE（Decoupled RoPE） · 第 53 页 ［原文 24］

23. MLA实际缓存什么 · 第 55 页 ［原文 25］

24. DeepSeek-V2的具体MLA尺寸 · 第 57 页 ［原文 26］

25. 为什么论文说 MLA 就像“有 2.25 个组的 GQA” · 第 59 页 ［原文 27］

26. 示例性 MHA/GQA/MQA/MLA 缓存比较 · 第 61 页 ［原文 28］

27. 93.3% DeepSeek-V2 数字，范围正确 · 第 63 页 ［原文 29］

28. DeepSeek 的消融作用做什么和不做什么 · 第 65 页 ［原文 30］

29. 四种并排的设计 · 第 67 页 ［原文 31］

30. FlashAttention 为独立执行轴 · 第 69 页 ［原文 32］

31. PagedAttention 是另一个不同的轴 · 第 71 页 ［原文 33］

32. 当前 PyTorch GQA 行为 · 第 73 页 ［原文 34］

33. 如何读取 模型（Model） 配置 · 第 75 页 ［原文 35］

34. 常见错误 · 第 77 页 ［原文 36］

35. 实用的决策框架 · 第 79 页 ［原文 37］

36. 与理解人工智能系列其余部分的联系 · 第 81 页 ［原文 38］

参考文献和进一步阅读 · 第 83 页 ［原文 39］

参考图：针对哪个问题阅读什么内容 · 第 85 页 ［原文 40］

了解 MHA、MQA、GQA 和 MLA

对应原文第 1 页；点击查看相邻原文对照页

了解 MHA、MQA、GQA 和 MLA 从 多头注意力（Multi-head Attention） 到潜在 KV 压缩 关于 查询（Query）、键（Key） 和 值（Value） 头结构如何改变 键值缓存（KV Cache）、解码带宽、模型容量（Model Capacity） 和现代 LLM 推理（Inference） 的技术手册 - 从标准多头注意力（Multi-head Attention）到多 查询（Query） 和 分组查询注意力（Grouped-query Attention） 到 DeepSeek 的 多头潜在注意力（Multi-head Latent Attention）。

Understanding MHA, MQA, GQA & MLA

From Multi-Head Attention to Latent KV Compression

A technical handbook on how query, key, and value head structure changes the KV cache, decode bandwidth, model capacity, and modern LLM inference - from standard multi-head attention through multi-query and grouped-query attention to DeepSeek's Multi-head Latent Attention.

@techNmak

如何阅读本手册

对应原文第 2 页；点击查看相邻原文对照页

如何阅读本手册 MHA、MQA、GQA 和 MLA 通常在图表中呈现为四个框。这种观点很有用，但它隐藏了这些设计存在的原因。重要的问题不仅仅是画出了多少个 头（Heads）。他们是：

1. 有多少个 查询头（Query Heads） 产生不同的 注意力机制（Attention） 分布？

2. 必须为每个先前的 词元（Token） 存储多少个不同的 键（Key）/值（Value） 表示？

3.自回归解码过程中实际缓存的表示（Representation）是什么？

对于 MHA、MQA 和 GQA，最简洁的 理解框架（Mental Model） 是将 查询头（Query Heads）的数量与键值头（Key/Value Heads）的数量分开。对于 MLA，模型（Model） 不再足够：主要的节省来自缓存低维 潜在表示（Latent Representation），而不是普通的完整每头 键（Keys） 和 值（Values）。[1][2][3][4] 核心区别。 MHA、GQA 和 MQA 主要可以通过 KV 头多重性来描述。 MLA 更改缓存的 表示（Representation） 本身。因此，将 MLA 称为“头（Heads） 更少的 GQA”是错误的 理解框架（Mental Model）。

本手册重点介绍 解码器（Decoder） 式自回归 推理（Inference），因为持久 KV-缓存容量（Cache Capacity） 和重复缓存读取变得尤为重要。 训练（Training） 行为将在区别重要的地方单独讨论。

符号 除非另有说明，否则我们使用 L 表示 注意力机制（Attention） 层数，T 表示缓存序列长度，b 表示每个缓存标量的字节数，HQ 表示 查询（Query） 头计数，HKV 表示 键（Key）/值（Value） 头计数，dh 表示普通每头 键（Key）/值（Value） 宽度。

对于 键（Key） 和 值（Value） 宽度相等的 MHA/MQA/GQA，一个序列的 缓存有效载荷（Cache Payload） 为

键（Keys） 和 值（Values） 的系数为 2。这是一个容量公式，而不是有关物理 HBM 流量的通用表述。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

How to read this handbook

MHA, MQA, GQA, and MLA are often presented as four boxes in a diagram. That view is useful, but it hides the reason these designs exist. The important questions are not just *how many heads are drawn*. They are:

1. **How many query heads produce distinct attention distributions?**
2. **How many distinct key/value representations must be stored for every previous token?**
3. **What representation is actually cached during autoregressive decoding?**

For MHA, MQA, and GQA, the cleanest mental model is to keep the number of query heads separate from the number of key/value heads. For MLA, that model stops being sufficient: the main saving comes from caching a low-dimensional latent representation rather than ordinary full per-head keys and values.[1][2][3][4]

> **Central distinction.** MHA, GQA, and MQA can be described primarily by *KV-head multiplicity*. MLA changes the *cached representation itself*. Calling MLA "GQA with even fewer heads" is therefore a false mental model.

This handbook focuses on decoder-style autoregressive inference because that is where persistent KV-cache capacity and repeated cache reads become especially important. Training behavior is discussed separately where the distinction matters.

Notation

We use L for the number of attention layers, T for cached sequence length, b for bytes per cached scalar, H_Q for query-head count, H_{KV} for key/value-head count, and d_h for the ordinary per-head key/value width unless stated otherwise.

For MHA/MQA/GQA with equal key and value widths, the cache payload for one sequence is

$$M_{\mathrm{KV}} = 2LTH_{KV}d_h b.$$

The factor of two is for keys and values. This is a capacity formula, not a universal statement about physical HBM traffic.

1. 从一个 注意力头（Attention Head） 开始

对应原文第 3 页；点击查看相邻原文对照页

1. 从一个 注意力头（Attention Head） 开始 对于解码步骤中的一个 因果自注意力（Causal Self-attention） 头，新的 queryqt 与位置 1, 的 键（Keys） 进行比较。 。 。 ， t。如果 kj, vj ∈ Rdh，则输出为

重要的 推理（Inference） 细节是时间不对称。新 词元（Token） 的 查询（Query） 使用一次。

当稍后生成 词元（Tokens） 时，将再次需要先前 词元（Tokens） 的 键（Keys） 和 值（Values）。

这就是为什么 解码器（Decoder） 推理（Inference） 将先前的 K/V 张量存储在 键值缓存（KV Cache） 中。

如果没有缓存，解码器（Decoder） 将重复重新计算整个前缀的 键（Keys） 和 值（Values）。通过缓存，模型（Model） 计算新 词元（Token） 的 K/V 一次，将其附加到缓存，并在后续步骤中重用存储的历史记录。

键值缓存（KV Cache） 不是在 注意力机制（Attention） 之外添加的可选“速度技巧”。这是在自回归解码期间避免重新计算过去的 键（Key）/值（Value） 投影的标准方法。因此，当上下文和批次增长时，它的大小就成为架构问题。

下一个问题是当 注意力机制（Attention） 有许多 头（Heads） 时会发生什么。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

1. Start with one attention head

For one causal self-attention head at decode step t, the new query $\boldsymbol{q}_t$ is compared with keys from positions $1, \ldots, t$. If $\boldsymbol{k}_j, \boldsymbol{v}_j \in \mathbb{R}^{d_h}$, the output is

$$\boldsymbol{o}_t = \sum_{j=1}^{t} p_{tj} \boldsymbol{v}_j, \qquad p_{tj} = \mathrm{softmax}_j\left(\frac{\boldsymbol{q}_t^\top \boldsymbol{k}_j}{\sqrt{d_h}}\right).$$

The important inference detail is temporal asymmetry. The query for the new token is used once. The keys and values for previous tokens will be needed again when later tokens are generated. That is why decoder inference stores previous K/V tensors in a KV cache.

Without caching, a decoder would repeatedly recompute keys and values for the whole prefix. With caching, the model computes the new token's K/V once, appends them to the cache, and reuses the stored history on subsequent steps.

> The KV cache is not an optional "speed trick" added outside attention. It is the standard way to avoid recomputing past key/value projections during autoregressive decoding. Its size therefore becomes an architectural concern when contexts and batches grow.

The next question is what happens when attention has many heads.

2、为什么存在多个头（Heads）

对应原文第 4 页；点击查看相邻原文对照页

2、为什么存在多个头（Heads） 原来的Transformer 模型（Transformer）是多个注意力头（Attention Heads）并联使用的。每个头接收自己的投影 查询（Queries）、键（Keys） 和 值（Values），计算 注意力机制（Attention） 结果，并且头输出被连接并投影回 模型（Model） 空间。 [1]

在常见的等宽情况下，

不同的 值（Value） 投影。

通常的解释是“不同的 头（Heads） 负责不同的事情”很直观但不完整。

从架构上来说，单独的 K/V 头（Heads） 也意味着解码期间单独的 K/V 投影参数和单独缓存的 K/V 向量（Vectors）。

第二个结果是 MQA 和 GQA 的动力。他们主要不是问多个 查询头（Query Heads） 是否有用。他们询问前缀中的每个 词元（Token） 是否每个 查询头（Query Head） 都必须拥有不同的 键（Key） 和 值（Value） 表示（Representation）。

不要将头数与 模型（Model） 宽度混为一谈。在许多架构中，dmodel = Hd h。

这是一种常见的设计选择，而不是数学定律。 DeepSeek-V2的MLA配置，

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

2. Why multiple heads exist

The original Transformer applies multiple attention heads in parallel. Each head receives its own projected queries, keys, and values, computes an attention result, and the head outputs are concatenated and projected back to the model space.[1]

In the common equal-width case,

$$\boldsymbol{q}_i = W_i^Q \boldsymbol{h}, \qquad \boldsymbol{k}_i = W_i^K \boldsymbol{h}, \qquad \boldsymbol{v}_i = W_i^V \boldsymbol{h},$$

for head $i \in \{1, \ldots, H\}$. Each head can learn a different query-key compatibility function and a different value projection.

The usual explanation that "different heads attend to different things" is intuitive but incomplete. Architecturally, separate K/V heads also mean separate K/V projection parameters and separate cached K/V vectors during decoding.

This second consequence is what motivates MQA and GQA. They do not primarily ask whether multiple query heads are useful. They ask whether every query head must own a distinct key and value representation for every token in the prefix.

Do not conflate head count with model width. In many architectures, $d_{\text{model}} = H d_h$. That is a common design choice, not a mathematical law. DeepSeek-V2's MLA configuration, for example, uses 128 attention heads of width 128 while the hidden dimension is 5120.[4]

3.多头注意力（Multi-head Attention）（MHA）

对应原文第 5 页；点击查看相邻原文对照页

3. 多头注意力（Multi-head Attention）(MHA) 在标准 MHA 中，查询（Query）、键（Key） 和 值（Value） 头数匹配：

对于每个 查询（Query） 头，都有一个相应的 键（Key） 头和 值（Value） 头。因此，在解码期间，每个先前的 词元（Token） 都会将 H 键（Key） 向量（Vectors） 和 H 值（Value） 向量（Vectors） 贡献给每个 注意力机制（Attention） 层的缓存。

跨所有 L 层的紧凑的每 词元（Token） 缓存计数为

标量。存储宽度b字节，

这种对 T 的线性依赖性很容易被低估。缓存随着序列长度 批大小（Batch Size）、层数、KV 头数 头宽度（Head Width） 和存储 精度（Precision） 的增长而增长。 模型（Model） 权重可能很合适，而缓存仍然限制并发长上下文请求的数量。

MHA 仍然是一个重要的参考点，因为它为每个 查询头（Query Head） 保留独立的 K/V 投影。但正是这种表征的丰富性才使其缓存变得如此之大。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

3. Multi-Head Attention (MHA)

In standard MHA, query, key, and value head counts match:

$$H_Q = H_{KV} = H.$$

For every query head i, there is a corresponding key head and value head. During decoding, every previous token therefore contributes H key vectors and H value vectors to each attention layer's cache.

A compact per-token cache count across all L layers is

$$N_{\text{MHA}} = 2LHd_h$$

scalars. With storage width b bytes,

$$M_{\text{MHA}} = 2LTHd_hb.$$

This linear dependence on T is easy to underestimate. The cache grows with sequence length, batch size, layer count, KV-head count, head width, and storage precision. The model weights may fit comfortably while the cache still constrains the number of concurrent long-context requests.

MHA remains an important reference point because it preserves independent K/V projections for every query head. But that representational richness is exactly what makes its cache large.

4. 训练（Training） 和解码强调不同的资源

对应原文第 6 页；点击查看相邻原文对照页

4. 训练（Training） 和解码强调不同的资源 在全序列 训练（Training） 期间，所有 词元（Token） 位置都是并行处理的。不存在必须在一次 词元（Token） 解码迭代中生存的长期 推理服务（Serving） 缓存。 注意力机制（Attention） 仍然消耗 激活（Activation） 内存和带宽，但持久 KV 缓存问题是 推理（Inference） 特别关注的问题。

在自回归解码过程中，每个步骤仅为每个序列创建一个新的 查询（Query） 位置，而 模型（Model） 会重复参与不断增长的前缀。先前计算的 K/V 张量必须可用于每个步骤。 Shazeer 的 MQA 论文将这些大 K/V 张量的重复加载确定为 增量解码（Incremental Decoding） 的主要内存带宽成本。 [2]

这就是为什么 MQA/GQA 最好理解为面向 推理（Inference） 的架构变更。他们还可以更改 训练（Training） 参数计数和投影工作，但其标志性好处是生成和缓存的不同 K/V 向量（Vectors） 更少。

解码 理解框架（Mental Model）.查询（Query） 的工作主要是“现在是新的 词元（Token）”。 KV 存储是“所有之前的 词元（Tokens），每一层，每个活动序列”。减少持久K/V 表示（Representation）攻击上下文长度和批大小（Batch Size）累积的部分。

4. Training and decoding stress different resources

During full-sequence training, all token positions are processed in parallel. There is no long-lived serving cache that must survive across one-token decode iterations. Attention still consumes activation memory and bandwidth, but the persistent KV-cache problem is specifically an inference concern.

During autoregressive decoding, each step creates only one new query position per sequence, while the model repeatedly attends over an ever-growing prefix. The previously computed K/V tensors must be available for every step. Shazeer's MQA paper identified the repeated loading of these large K/V tensors as a major memory-bandwidth cost of incremental decoding.[2]

This is why MQA/GQA are best understood as inference-oriented architectural changes. They can also change training parameter counts and projection work, but their signature benefit is that fewer distinct K/V vectors are produced and cached.

Decode mental model. Query work is mostly "new token now." KV storage is "all previous tokens, every layer, every active sequence." Reducing the persistent K/V representation attacks the part that accumulates with context length and batch size.

5. KV-cache公式，推导

对应原文第 7 页；点击查看相邻原文对照页

5. KV-cache公式，推导对于一层的一个词元（Token），假设有HKV 键头（Key Heads）和HKV 值头（Value Heads），每个宽度

键（Keys） 贡献 HKV dh 标量。 值（Values） 贡献另一个HKV dh。因此：

每层每个 词元（Token） 的标量 = 2HKV dh。

跨 L 层和 T 缓存的 词元（Token） 位置：

每个缓存标量有 b 字节：

对于缓存长度不等的批次T1，. 。 。 , TB，理想化的有效负载数为

实际的 推理服务（Serving） 系统可能会保留块、填充、页面、量化、卸载或维护元数据，因此分配的内存可能与此 张量（Tensor） 有效负载计算不同。

这个公式是架构和系统行为之间的桥梁：MHA、GQA 和 MQA 改变 HKV，同时通常保持 HQ 固定。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

5. The KV-cache formula, derived

For one token at one layer, suppose there are H_{KV} key heads and H_{KV} value heads, each of width d_h.

Keys contribute $H_{KV}d_h$ scalars. Values contribute another $H_{KV}d_h$. Therefore:

$$\text{scalars per token per layer} = 2H_{KV}d_h.$$

Across L layers and T cached token positions:

$$N_{\text{KV}} = 2LTH_{KV}d_h.$$

With b bytes per cached scalar:

$$\boxed{M_{\text{KV}} = 2LTH_{KV}d_hb.}$$

For a batch with unequal cached lengths $T_1, \ldots, T_B$, an idealized payload count is

$$M_{\text{KV}} = 2LH_{KV}d_hb\sum_{r=1}^{B} T_r.$$

Actual serving systems may reserve blocks, pad, page, quantize, offload, or maintain metadata, so allocated memory can differ from this tensor-payload calculation.

This formula is the clean bridge between architecture and systems behavior: MHA, GQA, and MQA change H_{KV} while often keeping H_Q fixed.

6. 解码带宽：为什么 键值头（KV Heads） 帮助较少

对应原文第 8 页；点击查看相邻原文对照页

6. 解码带宽：为什么更少的 键值头（KV Heads） 有帮助解码步骤中一层的简单有效负载 模型（Model） 与 2T HKV dhb 成正比，因为新的 查询（Query） 必须与缓存前缀中的 键（Keys） 和 值（Values） 交互。这不是通用的 HBM 字节计数：真正的 内核（Kernels） 利用缓存层次结构、分块（Tiling）、批处理（Batching）、矢量化和重用。但它暴露了架构杠杆。

如果 32 个 查询头（Query Heads） 各自拥有不同的 K/V 头（Heads），则缓存中每个 词元（Token） 包含 32 K 头（Heads） 和 32 V 头（Heads）。如果相同的 32 个 查询头（Query Heads） 仅共享 8 K/V 头（Heads），则持久 缓存有效载荷（Cache Payload） 的大小是四分之一。如果所有32个查询（Queries）共享一个K/V头，那么在相同的L、T、dh、b假设下，它是MHA缓存的三十秒大。

注意力机制（Attention） 分数算术则不同。仍有 32 个 查询头（Query Heads） 产生 32 个 注意力机制（Attention） 输出。共享 K/V 头不会删除那些 查询（Query） 头计算。它主要减少 K/V 投影多重性、缓存容量（Cache Capacity） 以及需要提供给 注意力机制（Attention） 的不同缓存 K/V 数据量。

这就是为什么“GQA 将 注意力机制（Attention） 复杂性降低了 4 倍”通常过于模糊而无用。

哪种复杂性？在 32Q/8KV 示例中，K/V 投影工作和 缓存有效载荷（Cache Payload） 可能会下降 4 倍，而密集的 查询（Query）-键（Key） 得分仍然有 32 个 查询头（Query Heads）。

6. Decode bandwidth: why fewer KV heads help

A simple payload model for one layer at a decode step is proportional to

$$2TH_{KV}d_hb,$$

because the new query must interact with keys and values from the cached prefix. This is not a universal HBM-byte count: real kernels exploit cache hierarchy, tiling, batching, vectorization, and reuse. But it exposes the architectural lever.

If 32 query heads each own distinct K/V heads, the cache contains 32 K heads and 32 V heads per token. If the same 32 query heads share only 8 K/V heads, the persistent cache payload is one quarter as large. If all 32 queries share one K/V head, it is one thirty-second as large as the MHA cache under the same L, T, d_h, b assumptions.

The attention-score arithmetic is a different story. There are still 32 query heads producing 32 attention outputs. Sharing a K/V head does not erase those query-head computations. It primarily reduces K/V projection multiplicity, cache capacity, and the amount of distinct cached K/V data that needs to be supplied to attention.

> This is why “GQA reduces attention complexity by 4x” is usually too vague to be useful. Which complexity? K/V projection work and cache payload can fall by 4x in a 32Q/8KV example, while dense query-key scoring still has 32 query heads.

7.多查询注意力（Multi-query Attention）（MQA）

对应原文第 9 页；点击查看相邻原文对照页

7. 多查询注意力（Multi-query Attention）(MQA) 多查询注意力（Multi-query Attention） 保留多个 查询头（Query Heads），但共享一个 键（Key） 头和一个 值（Value） 头。 Shazeer 引入该变体专门是为了降低 K/V 张量（Tensor） 大小和 增量解码（Incremental Decoding） 的内存带宽要求。[2]

定义计数是

每个 查询头（Query Head） 仍具有自己的 查询（Query） 投影，因此可以生成不同的 注意力分布（Attention Distribution）。共享的是键（Key）和值（Value）侧前缀的表示（Representation）。

对于 查询（Query） 头，

t,ikj

每个 查询头（Query Head） 都会读取相同的 kj 和 vj。

MQA 并不是指一个 注意力头（Attention Head），而是指一对 KV 头。 模型（Model） 仍然可以有许多独立的 查询头（Query Heads） 和许多不同的 注意力机制（Attention） 分布。

这种区别是 MQA 最常见的实现和解释错误之一。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

7. Multi-Query Attention (MQA)

Multi-Query Attention keeps multiple query heads but shares one key head and one value head across all of them. Shazeer introduced the variant specifically to reduce the K/V tensor size and memory-bandwidth requirements of incremental decoding.[2]

The defining counts are

$$H_Q = H, \qquad H_{KV} = 1.$$

Each query head still has its own query projection and therefore can produce a different attention distribution. What is shared is the representation of the prefix on the key and value side.

For query head i,

$$\boldsymbol{o}_{t,i} = \sum_{j \le t} \mathrm{softmax}_j\left(\frac{\boldsymbol{q}_{t,i}^\top \boldsymbol{k}_j}{\sqrt{d_h}}\right) \boldsymbol{v}_j.$$

The same $\boldsymbol{k}_j$ and $\boldsymbol{v}_j$ are read by every query head.

> **MQA does not mean one attention head.** It means one *KV head pair*. The model can still have many independent query heads and many distinct attention distributions.

That distinction is one of the most common implementation and explanation errors around MQA.

8. MQA 能节省什么，不能节省什么

对应原文第 10 页；点击查看相邻原文对照页

8. MQA 保存了什么 - 以及不保存什么 在 MHA/MQA 缓存公式下，将 H 键值头（KV Heads） 替换为 1 会更改 2LT Hdhb → 2LT dhb。

如果所有其他项保持相等，则高速缓存缩减因子为 H。

MQA 还缩小了 K 和 V 投影输出宽度，通常还缩小了它们的参数计数。

然而：

- 查询头（Query Heads）的数量无需更改；

- 输出在输出投影（Output Projection） 之前仍包含每个查询头（Query Head） 的一个结果；

- 稠密注意力（Dense Attention） 仍将每个 查询头（Query Head） 与出席的 词元（Token） 位置进行比较；

- 序列长度依赖性不会神奇地消除。

原始 MQA 论文报告在其实验设置中解码速度要快得多，相对于 基线（Baseline），质量仅略有下降。[2] 该经验结果不应该被推广为 MQA 始终是质量中立的定理。后来的工作，包括 GQA 和 DeepSeek-V2 研究，提供了一些例子，在其测试设置中，与 MHA 相比，更积极的 KV 共享会损失质量。[3][4]因此，该架构暴露了一个真正的权衡：极端的 K/V 共享可以非常高效地缓存，但它也减少了 查询头（Query Heads）可用的不同学习 K/V 子空间的数量。

8. What MQA saves - and what it does not

Under the MHA/MQA cache formula, replacing H KV heads with one changes

$$2LTHd_hb \quad \longrightarrow \quad 2LTd_hb.$$

The cache reduction factor is H if all other terms remain equal.

MQA also shrinks the K and V projection output widths and usually their parameter counts. However:

- the number of query heads need not change;
- the output still contains one result per query head before the output projection;
- dense attention still compares every query head with the attended token positions;
- sequence-length dependence is not magically removed.

The original MQA paper reported much faster decoding in its experimental setting with only minor quality degradation relative to its baseline.[2] That empirical result should not be promoted into a theorem that MQA is always quality-neutral. Later work, including the GQA and DeepSeek-V2 studies, provides examples where more aggressive KV sharing loses quality relative to MHA in their tested settings.[3][4]

The architecture therefore exposes a real trade-off: extreme K/V sharing can be exceptionally cache-efficient, but it also reduces the number of distinct learned K/V subspaces available to the query heads.

9.分组查询注意力（Grouped-query Attention）（GQA）

对应原文第 11 页；点击查看相邻原文对照页

9. 分组查询注意力（Grouped-query Attention）(GQA) GQA 在MHA 和MQA 之间插入中间点。它使用键值头（KV Heads）的一些中间数字G来代替H 键值头（KV Heads）或一个KV头。 查询头（Query Heads） 分为多个组，一组中的所有 查询（Queries） 共享该组的 键（Key） 和 值（Value） 头。 [3]

让

并假设 H 可被 G 整除。每个KV组的查询头（Query Heads）数量为

一个简单的组图是

所以 查询（Query） headi 使用 键（Key）/值（Value） headg(i) 参加。

输出变为

这使得连续统精确：

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

9. Grouped-Query Attention (GQA)

GQA inserts intermediate points between MHA and MQA. Instead of H KV heads or one KV head, it uses some intermediate number G of KV heads. Query heads are divided into groups, and all queries in one group share that group's key and value head.[3]

Let

$$H_Q = H, \qquad H_{KV} = G, \qquad 1 < G < H,$$

and assume H is divisible by G. The number of query heads per KV group is

$$r = \frac{H}{G}.$$

A simple group map is

$$g(i) = 1 + \left\lfloor \frac{i-1}{r} \right\rfloor,$$

so query head i attends using key/value head $g(i)$.

The output becomes

$$\boldsymbol{o}_{t,i} = \sum_{j \le t} \mathrm{softmax}_j \left(\frac{\boldsymbol{q}_{t,i}^\top \boldsymbol{k}_{j,g(i)}}{\sqrt{d_h}} \right) \boldsymbol{v}_{j,g(i)}.$$

This makes the continuum exact:

$$G = H \Rightarrow \text{MHA}, \qquad 1 < G < H \Rightarrow \text{GQA}, \qquad G = 1 \Rightarrow \text{MQA}.$$

10. MHA、GQA 和 MQA 一张图

对应原文第 12 页；点击查看相邻原文对照页

10. MHA、GQA、MQA MHA 一张图：8Q / 8KV

质量保证：8Q/2KV

MQA：8Q/1KV

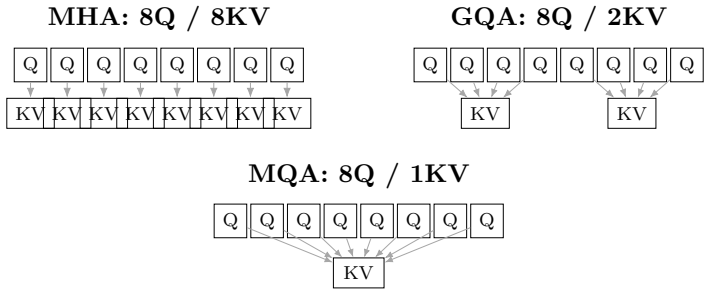
该图特意为配对的 键值头（Key/Value Head） 绘制了一个“KV”框。所有三个示例中 Q 框的数量均保持不变。仅不同 KV 对的数量发生变化。

对于固定的dh，缓存有效载荷（Cache Payload）因此分别与8、2和1成比例。但在每种情况下，查询（Query） 头输出的数量仍然是八个。

这张图是读取现代 模型（Model） 配置的最简单可靠的方法：分别识别 查询（Query） 头数和 KV 头数。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

10. One diagram for MHA, GQA, and MQA



The diagram intentionally draws a single “KV” box for a paired key/value head. The number of Q boxes is unchanged across all three examples. Only the number of distinct KV pairs changes.

For a fixed d_h, the cache payload is therefore proportional to 8, 2, and 1 respectively. But the number of query-head outputs remains eight in every case.

This picture is the simplest reliable way to read a modern model config: identify the query-head count and the KV-head count separately.

11. GQA 也是一个向上训练的良方

对应原文第 13 页；点击查看相邻原文对照页

11. GQA 也是一种向上训练的方法 GQA 论文的作用不仅仅是命名一个中间架构。它询问是否可以将现有的 MHA 检查点转换为 MQA 或 GQA，而无需从头开始重新训练。 [3]

为了进行转换，对每个目标组内的 K/V 头（Heads） 进行均值池化以初始化共享 K/V 头。转换后的 模型（Model） 然后在 预训练（Pretraining） 中继续。该论文的标题实验仅使用了原始 预训练（Pretraining） 计算量的 5% 用于此上训练阶段。[3]

这个历史细节很重要，因为有两个想法经常被推翻：

- GQA 作为架构：键值头（KV Heads） 比 查询头（Query Heads） 少。

- GQA 上行训练：将MHA 检查点转换为该架构的特殊方法。

模型（Model） 可以从头开始使用 GQA 进行训练；它不必源自 MHA 检查点。相反，mean-池化（Pooling） 初始化是转换配方的一部分，而不是 分组查询注意力（Grouped-query Attention） 的数学定义。

当前的 Hugging Face Llama 配置文档仍然描述了这种mean-池化（Pooling） 转换约定，用于从 MHA 检查点构造分组 K/V 头（Heads）。 [7]

11. GQA was also an uptraining recipe

The GQA paper did more than name an intermediate architecture. It asked whether an existing MHA checkpoint could be converted into MQA or GQA without retraining from scratch.[3]

For conversion, K/V heads within each target group are mean-pooled to initialize the shared K/V head. The converted model is then continued in pretraining. The paper's headline experiments used only 5% of the original pretraining compute for this uptraining stage.[3]

This historical detail matters because two ideas are often collapsed:

- **GQA as an architecture:** fewer KV heads than query heads.
- **GQA uptraining:** a particular way to convert an MHA checkpoint into that architecture.

A model can be trained with GQA from scratch; it does not have to originate from an MHA checkpoint. Conversely, the mean-pooling initialization is part of the conversion recipe, not the mathematical definition of grouped-query attention.

Current Hugging Face Llama configuration documentation still describes this mean-pooling conversion convention for constructing grouped K/V heads from an MHA checkpoint.[7]

12. 原始 GQA 结果实际显示了什么

对应原文第 14 页；点击查看相邻原文对照页

12. 原始 GQA 结果实际上显示了什么 GQA 论文被引用最多的结果来自经过转换和上训练的 T5-XXL 模型（Models）

模型（Model） 时间/样本平均值得分解读 MHA-XXL 1.51 47.2 质量参考 MQA-XXL 0.24 46.6 这三个中最快的 GQA-8-XXL 0.28 47.1 接近 MQA 时间，接近 MHA 平均 平均值结合了论文的摘要、翻译和问答评估集；指标因任务而异。上表是对论文报道结果的有意精简选择，并非新的通用基准测试（Benchmark）。

正确的结论是狭窄的：在该实验中，GQA-8 恢复了 MHA 基线（Baseline） 的大部分平均质量，同时保留了接近 MQA 的 推理（Inference） 时间。该论文并未证明每个 仅解码器大语言模型（Decoder-only LLM）、内核（Kernel）、批大小（Batch Size） 或硬件平台都会表现出相同的质量-速度曲线。

这种范围性阅读比“GQA 以 MQA 速度提供 MHA 质量”这一口号更有用。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

12. What the original GQA result actually showed

The GQA paper's most cited result came from T5-XXL models that were converted and uptrained for 5% of original pretraining compute. In that setup, the authors reported:[3]

Model	Time/sample (s)	Avg. score	Interpretation
MHA-XXL	1.51	47.2	quality reference
MQA-XXL	0.24	46.6	fastest of these three
GQA-8-XXL	0.28	47.1	near MQA time, near MHA average

The average combines the paper's summarization, translation, and question-answering evaluation set; the metrics differ by task. The table above is a deliberately reduced selection of the paper's reported results, not a new universal benchmark.

The correct takeaway is narrow: **in that experiment**, GQA-8 recovered most of the MHA baseline's average quality while retaining inference time close to MQA. The paper did not prove that every decoder-only LLM, kernel, batch size, or hardware platform will exhibit the same quality-speed curve.

That scoped reading is more useful than the slogan "GQA gives MHA quality at MQA speed."

13. 32Q / 8KV 分组示例

对应原文第 15 页；点击查看相邻原文对照页

13. 一个有效的 32Q / 8KV 分组示例 假设一个层有

然后各个KV头服务

查询头（Query Heads）。

一个自然的分组是：

等等通过KV8。

对于一层的一个 词元（Token）：

- 查询（Query） 表示（Representation） 宽度：32× 128 = 4096 个标量；

- 键（Key） 缓存宽度：8×128 = 1024 标量；

- 值（Value） 缓存宽度：8×128 = 1024 标量；

- 总缓存K/V 宽度：2048 标量。

在具有 32 键值头（KV Heads） 的 MHA 下，缓存宽度将为 8192 标量。因此，在此示例中，GQA 将原始 K/V 缓存有效载荷（Cache Payload） 减少了 4 倍，同时保留了 32 个 查询头（Query Heads）。

最后一句话是重点：模型（Model） 仍然有 32 个 查询头（Query Heads）。 GQA 改变了 查询（Queries） 参考的不同 K/V 表示的数量。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

13. A worked 32Q / 8KV grouping example

Suppose a layer has

$$H_Q = 32, \qquad H_{KV} = 8, \qquad d_h = 128.$$

Then each KV head serves

$$r = \frac{32}{8} = 4$$

query heads.

A natural grouping is:

$$\{Q_1, Q_2, Q_3, Q_4\} \to KV_1,$$
$$\{Q_5, Q_6, Q_7, Q_8\} \to KV_2,$$

and so on through KV_8.

For one token at one layer:

- query representation width: $32 \times 128 = 4096$ scalars;
- key cache width: $8 \times 128 = 1024$ scalars;
- value cache width: $8 \times 128 = 1024$ scalars;
- total cached K/V width: 2048 scalars.

Under MHA with 32 KV heads, the cached width would be 8192 scalars. GQA therefore reduces the raw K/V cache payload by 4x in this example while retaining 32 query heads.

That last phrase is the point: the model still has 32 query heads. GQA changes how many distinct K/V representations those queries consult.

14. 工作缓存计算：MHA vs GQA vs MQA

对应原文第 16 页；点击查看相邻原文对照页

14. 有效缓存计算：MHA vs GQA vs MQA 在所有变体中使用相同的配置：

相对于 MHA 的变体 键值头（KV Heads） 缓存 MHA 32 4 GiB 1x GQA 8 1 GiB 1/4 MQA 1 128 MiB 1/32 这些是一个序列的 张量（Tensor） 有效负载编号。它们不包括分配器元数据、碎片化（Fragmentation）、其他 激活（Activations）、模型（Model） 权重、临时工作区和特定于框架的存储。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

14. Worked cache calculation: MHA vs GQA vs MQA

Use the same configuration across all variants:

$$L = 32, \quad T = 8192, \quad H_Q = 32, \quad d_h = 128, \quad b = 2 \text{ bytes}.$$

MHA: $H_{KV} = 32$

$$M = 2 \cdot 32 \cdot 8192 \cdot 32 \cdot 128 \cdot 2 = 4{,}294{,}967{,}296 \text{ bytes} = 4 \text{ GiB}.$$

GQA: $H_{KV} = 8$

$$M = 1{,}073{,}741{,}824 \text{ bytes} = 1 \text{ GiB}.$$

MQA: $H_{KV} = 1$

$$M = 134{,}217{,}728 \text{ bytes} = 128 \text{ MiB}.$$

Variant	KV heads	Cache	Relative to MHA
MHA	32	4 GiB	1x
GQA	8	1 GiB	1/4
MQA	1	128 MiB	1/32

These are tensor-payload numbers for one sequence. They exclude allocator metadata, fragmentation, other activations, model weights, temporary workspace, and framework-specific storage.

15. 磁头共享的变化大于内存容量的变化

对应原文第 17 页；点击查看相邻原文对照页

15. 磁头共享变化大于内存容量减少 HKV 会影响 模型（Model） 的多个部分：

K/V 投影宽度。在传统实现中，键值头（KV Heads） 越少意味着 K 和 V 投影输出越窄。因此，当 头宽度（Head Width） 固定时，它们的参数计数和投影工作可以大致与 HKV 成比例下降。

缓存容量（Cache Capacity）。持久的每 词元（Token） 缓存随 HKV 线性扩展。

解码期间读取不同的 缓存有效载荷（Cache Payload）。较少的唯一 KV 向量（Vectors） 可以减少必须从内存提供的 K/V 数据量，特别是当 内核（Kernels） 跨多个 查询（Queries） 重用共享 KV 头时。

代表性共享。多个 查询头（Query Heads） 现在询问相同的 K/V 表示（Representation）。

这是一个架构约束，而不仅仅是系统优化。

完整的 稠密注意力（Dense Attention） 算术不会自动下降相同的系数。每个 查询头（Query Head） 仍然形成自己的 注意力机制（Attention） 分数和加权输出。确切的运行时影响取决于预测、注意力机制（Attention） 数学、内存流量（Memory Traffic）、内核（Kernel） 设计、批大小（Batch Size） 和硬件的相对成本。

15. Head sharing changes more than memory capacity

Reducing H_{KV} affects several parts of the model:

K/V projection width. In a conventional implementation, fewer KV heads mean narrower K and V projection outputs. Their parameter count and projection work can therefore fall roughly in proportion to H_{KV} when head width is fixed.

Cache capacity. The persistent per-token cache scales linearly with H_{KV}.

Distinct cache payload read during decode. Fewer unique KV vectors can reduce the amount of K/V data that must be supplied from memory, especially when kernels reuse a shared KV head across multiple queries.

Representational sharing. Multiple query heads now interrogate the same K/V representation. This is an architectural constraint, not just a systems optimization.

What does not automatically fall by the same factor is the full dense attention arithmetic. Every query head still forms its own attention scores and weighted output. The exact runtime impact depends on the relative costs of projections, attention math, memory traffic, kernel design, batch size, and hardware.

16. 现代模型（Models）使GQA成为主流

对应原文第 18 页；点击查看相邻原文对照页

16. 现代 模型（Models） 使 GQA 成为主流 GQA 迅速从转换技术转变为标准 模型（Model） 设计选择。Llama 2 在其较大的 34B 和 70B 模型（Models） 中使用了 GQA，以提高 推理（Inference） 可扩展性。[8] Mistral 7B 还明确采用了 GQA 以获得更快的 推理（Inference）。[9]随后，Meta 在 8B 和 70B Llama 3 尺寸上使用了 GQA。[10]

这些例子与其说是一场流行竞赛，不如说是设计压力的 证据（Evidence）：长上下文、更大的批次和高吞吐量（Throughput） 推理服务（Serving） 使得在每个 查询头（Query Head） 上复制 K/V 状态的成本越来越明显。

尽管如此，“现代 LLM = GQA”还是太宽泛了。架构继续探索其他策略，包括潜在缓存压缩（例如 MLA）以及其他形式的 KV 压缩或驱逐。

重要的教训是，KV 表示（Representation） 现在是一流的架构决策。

模型（Model） 示例是实现事实，而不是定义。无论哪个名为 模型（Models） 采用它，GQA 仍然是 GQA，并且 模型（Model） 系列可以跨尺寸或跨代更改 注意力机制（Attention） 设计。

16. Modern models made GQA mainstream

GQA moved quickly from a conversion technique into a standard model-design choice. Llama 2 used GQA in its larger 34B and 70B models for improved inference scalability.[8] Mistral 7B also explicitly adopted GQA for faster inference.[9] Meta then used GQA across both the 8B and 70B Llama 3 sizes.[10]

These examples matter less as a popularity contest than as evidence of the design pressure: long contexts, larger batches, and high-throughput serving make the cost of duplicating K/V state across every query head increasingly visible.

Still, "modern LLM = GQA" would be too broad. Architectures continue to explore other strategies, including latent cache compression such as MLA and additional forms of KV compression or eviction. The important lesson is that KV representation is now a first-class architectural decision.

> Model examples are implementation facts, not definitions. GQA remains GQA regardless of which named models adopt it, and model families can change attention designs across sizes or generations.

17. MLA 是设计空间的一个不同分支

对应原文第 19 页：点击查看相邻原文对照页

17. MLA 是设计空间 MQA 和 GQA 的不同分支，要求：

应存储多少个常规 K/V 头（Heads）？

多头潜在注意力（Multi-head Latent Attention） 提出了一个不同的问题：

我们是否需要存储传统的每头 K/V 张量？

DeepSeek-V2引入了具有键（Keys）和值（Values）低秩联合压缩的MLA。 隐藏状态（Hidden State） 不是为每个头和 词元（Token） 缓存完整的 键（Key） 向量（Vector） 和完整的 值（Value） 向量（Vector），而是将 隐藏状态（Hidden State） 投影到紧凑的潜在 向量（Vector） 中，从中导出内容 K/V 表示。 [4]

这就是为什么 MLA 不应放置在标记为“键值头（KV Heads） 的数量”的简单一维轴上。它的主要压缩参数是潜在宽度dc，而不是 HKV 。

该设计还有一个单独的小型位置 键（Key） 组件，因为普通 RoPE 会干扰使潜在缓存高效的代数 投影吸收（Projection Absorption）。这种互动是理解 MLA 的核心，而不是实施脚注。

17. MLA is a different branch of the design space

MQA and GQA ask:

> How many conventional K/V heads should be stored?

Multi-head Latent Attention asks a different question:

> Do we need to store the conventional per-head K/V tensors at all?

DeepSeek-V2 introduced MLA with low-rank *joint* compression of keys and values. Instead of caching a full key vector and full value vector for every head and token, the hidden state is projected into a compact latent vector from which the content K/V representations are derived.[4]

This is why MLA should not be placed on a simple one-dimensional axis labeled "number of KV heads." Its main compression parameter is the latent width d_c, not H_{KV}.

The design also has a separate small positional key component because ordinary RoPE interferes with the algebraic projection absorption that makes latent caching efficient. That interaction is central to understanding MLA rather than an implementation footnote.

18.低秩联合K/V压缩

对应原文第 20 页；点击查看相邻原文对照页

18.低秩联合K/V压缩令ht∈Rdmodel为tokent的层输入。 MLA首先将其压缩为共享K/V潜在cKV

d ht, cKV

然后可以从该潜在变量生成内容 键（Keys） 和 值（Values）：

UcKV

UcKV

DeepSeek-V2 使用比全级联多头 K 或 V 宽度小得多的 dc。[4]

短语“低秩”应该相对于大的完整 K/V 投影空间来解释。潜伏不一定小于一个个体 头维度（Head Dimension）。例如，在 DeepSeek-V2 中，dh = 128，而 dc = 512。重要的是 512 远低于完整的 128 头内容

在优化 推理（Inference） 期间，latentcKV t 是可重用内容状态。这是与普通 MHA/GQA/MQA 缓存的结构突破。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

18. Low-rank joint K/V compression

Let $\boldsymbol{h}_t \in \mathbb{R}^{d_{\text{model}}}$ be the layer input for token t. MLA first compresses it into a shared K/V latent

$$\boldsymbol{c}_t^{KV} = W_D^{KV} \boldsymbol{h}_t, \qquad \boldsymbol{c}_t^{KV} \in \mathbb{R}^{d_c}.$$

Content keys and values can then be generated from that latent:

$$\boldsymbol{k}_t^C = W_U^K \boldsymbol{c}_t^{KV}, \qquad \boldsymbol{v}_t^C = W_U^V \boldsymbol{c}_t^{KV}.$$

DeepSeek-V2 uses d_c much smaller than the full concatenated multi-head K or V width.[4]

The phrase *low-rank* should be interpreted relative to the large full K/V projection space. The latent does not have to be smaller than one individual head dimension. In DeepSeek-V2, for example, $d_h = 128$ while $d_c = 512$. What matters is that 512 is far below the full 128-head content width $128 \times 128 = 16{,}384$.[4]

During optimized inference, the latent $\boldsymbol{c}_t^{KV}$ is the reusable content state. That is the structural break from ordinary MHA/GQA/MQA caching.

19.吸收技巧：不要仅仅为了缓存而重建完整的K/V

对应原文第 21 页；点击查看相邻原文对照页

19. 吸收技巧：不要仅仅为了缓存而重建完整的K/V 乍一看，潜在压缩（Latent Compression） 似乎只是通过添加解压工作来节省存储空间：每次需要时都从 cKV 重建 kC 和 vC。 MLA 通过 矩阵乘法（Matrix Multiplication） 的关联性避免了这种幼稚的实现。[4]

对于一个头，内容分数包含

U,icKV

将其重新关联为 (

键（Key） 升维投影（Up-projection） 已有效移动到 查询（Query） 一侧。缓存的前缀 词元（Tokens） 可以保留在潜在空间中。

对于 值（Values），

pij(W V U,icKV

pijcKV

值（Value） 升维投影（Up-projection） 可以在加权潜在聚合之后应用，并与周围的输出侧线性图组合。

实际的想法不是“压缩，然后完全解压缩每个缓存的 词元（Token）”。它是重写周围的线性代数，使注意力机制（Attention）可以与压缩潜在表示（Compressed Latent）一起运算，同时固定的投影矩阵（Projection Matrices）被吸收到相邻的计算中。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

19. The absorption trick: do not reconstruct full K/V just to cache them

At first glance, latent compression seems to save storage only by adding decompression work: reconstruct $\boldsymbol{k}^C$ and $\boldsymbol{v}^C$ from $\boldsymbol{c}^{KV}$ every time they are needed. MLA avoids that naive implementation through associativity of matrix multiplication.[4]

For one head, the content score contains

$$(\boldsymbol{q}_i^C)^\top (W_{U,i}^K \boldsymbol{c}_j^{KV}).$$

Reassociate it as

$$((W_{U,i}^K)^\top \boldsymbol{q}_i^C)^\top \boldsymbol{c}_j^{KV}.$$

The key up-projection is effectively moved to the query side. Cached prefix tokens can remain in latent space.

For values,

$$\sum_j p_{ij} (W_{U,i}^V \boldsymbol{c}_j^{KV}) = W_{U,i}^V \left(\sum_j p_{ij} \boldsymbol{c}_j^{KV} \right).$$

The value up-projection can be applied after the weighted latent aggregation and combined with surrounding output-side linear maps.

> The practical idea is not "compress, then fully decompress every cached token." It is to rewrite the surrounding linear algebra so attention can operate with the compressed latent while fixed projection matrices are absorbed into adjacent computation.

20. 查询（Query） 压缩是一个单独的想法

对应原文第 22 页；点击查看相邻原文对照页

20. 查询（Query） 压缩是一个单独的想法 DeepSeek-V2 还在 查询（Query） 路径上应用了低秩投影：

该论文的目的是减少 训练（Training） 期间的 激活（Activation） 内存。它明确指出 查询（Query） 压缩不会减少 推理（Inference） 键值缓存（KV Cache）。[4]

这很直观。 查询（Queries） 是为当前 词元（Token） 生成的，不是未来解码步骤需要保留的持久历史记录。压缩 查询（Query） 路径可以更改 训练（Training） 激活（Activation） 存储和参数化，但长期存在的 推理服务（Serving） 缓存位于 键（Key）/值（Value） 一侧。

这种区别在 MLA 之外很有用：在评估任何“注意力机制（Attention） 压缩”声明时，询问正在压缩什么以及 张量（Tensor） 何时生效？较小的临时 查询（Query） 激活（Activation） 和较小的持久 键值缓存（KV Cache） 可以解决不同的系统问题。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

20. Query compression is a separate idea

DeepSeek-V2 also applies a low-rank projection on the query path:

$$\boldsymbol{c}_t^Q = W_D^Q \boldsymbol{h}_t, \qquad \boldsymbol{q}_t^C = W_U^Q \boldsymbol{c}_t^Q.$$

The paper's stated motivation is to reduce activation memory during training. It explicitly notes that query compression does *not* reduce the inference KV cache.[4]

That makes intuitive sense. Queries are produced for the current token and are not the persistent history that future decode steps need to retain. Compressing the query pathway can change training activation storage and parameterization, but the long-lived serving cache is on the key/value side.

This distinction is useful beyond MLA: when evaluating any "attention compression" claim, ask *what is being compressed and when does that tensor live?* A smaller temporary query activation and a smaller persistent KV cache solve different systems problems.

21. 为什么普通RoPE会破坏简单的MLA吸收

对应原文第 23 页；点击查看相邻原文对照页

21. 为什么普通RoPE会破坏简单的MLA吸收 RoPE对查询（Queries）和键（Keys）应用位置相关的旋转。在普通的 注意力机制（Attention） 中，这正是我们想要的：查询（Query）-键（Key） 兼容性取决于旋转的相对位置。

但是，在 线性投影（Linear Projections） 之间插入位置相关的 算子（Operator） 会更改 MLA 可用的代数。[5]

如果没有位置旋转，则可以重新关联内容分数，以便固定的 键（Key） 升维投影（Up-projection） 移动到 查询（Query） 一侧。使用重建内容 键（Key） 上的 RoPE，分数示意性地包含 查询（Query） 侧 向量（Vector） 和 键（Key） 升维投影（Up-projection） 之间的位置相关变换：

t RjW K U ckV

因为 Rj 取决于缓存的 词元（Token） 位置 j，所以不存在可以简单地吸收所有前缀位置的 W K U 的单个位置无关的 矩阵（Matrix）。

DeepSeek-V2 将此描述为标准 RoPE 放置与所需的低阶 KV 吸收之间的不兼容。在每个解码步骤中为整个前缀重建特定于位置的完整 键（Keys） 将抵消 推理（Inference） 的大部分好处。 [4]

解决方案是将位置信息保留在单独的较小路径上。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

21. Why ordinary RoPE breaks simple MLA absorption

RoPE applies a position-dependent rotation to queries and keys. In ordinary attention that is exactly what we want: query-key compatibility depends on relative position through the rotations. But a position-dependent operator inserted between linear projections changes the algebra available to MLA.[5]

Without positional rotation, a content score can be reassociated so the fixed key up-projection moves to the query side. With RoPE on the reconstructed content key, the score schematically contains a position-dependent transform between the query-side vector and the key up-projection:

$$\boldsymbol{q}_t^\top R_t^\top R_j W_U^K \boldsymbol{c}_j^{KV}.$$

Because R_j depends on the cached token position j, there is no single position-independent matrix that can simply absorb W_U^K for all prefix positions.

DeepSeek-V2 describes this as an incompatibility between standard RoPE placement and the desired low-rank KV absorption. Reconstructing position-specific full keys for the entire prefix at every decode step would defeat much of the inference benefit.[4]

The solution is to keep positional information on a separate, smaller path.

22.解耦 RoPE（Decoupled RoPE）

对应原文第 24 页；点击查看相邻原文对照页

22. 解耦 RoPE（Decoupled RoPE） MLA将键（Key）/查询（Query） 表示（Representation）分为内容部分和RoPE承载部分。对于 查询头（Query Head） i：

而 键（Key） 是

RoPE 查询（Query） 组件 $q_{R\ t,i}$ 是头特定的，但 RoPE 键（Key） 组件 $k_{R\ j}$ 在 DeepSeek-V2 结构中的 头（Heads） 之间共享。 [4]

因此，注意力机制（Attention） 分数结合了内容和位置兼容性：

只有小型共享 RoPE 键（Key） 必须与 K/V 潜伏一起添加到持久缓存中。

大内容 键（Keys） 可以通过 投影吸收（Projection Absorption） 保持隐式。

此设计是架构协同设计的一个清晰示例：修改 位置编码（Positional Encoding） 并不是因为 RoPE“不好”，而是因为它通常的放置会阻止架构想要利用的 推理（Inference） 优化。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

22. Decoupled RoPE

MLA separates the key/query representation into a content part and a RoPE-carrying part. For query head i:

$$\boldsymbol{q}_{t,i} = [\boldsymbol{q}^C_{t,i}; \boldsymbol{q}^R_{t,i}],$$

while the key is

$$\boldsymbol{k}_{j,i} = [\boldsymbol{k}^C_{j,i}; \boldsymbol{k}^R_{j}].$$

The RoPE query component $\boldsymbol{q}^R_{t,i}$ is head-specific, but the RoPE key component $\boldsymbol{k}^R_j$ is shared across heads in the DeepSeek-V2 construction.[4]

The attention score therefore combines content and positional compatibility:

$$\frac{(\boldsymbol{q}^C_{t,i})^\top \boldsymbol{k}^C_{j,i} + (\boldsymbol{q}^R_{t,i})^\top \boldsymbol{k}^R_j}{\sqrt{d_h + d^R_h}}.$$

Only the small shared RoPE key must be added to the persistent cache alongside the K/V latent. The large content keys can remain implicit through projection absorption.

This design is a clean example of architectural co-design: the positional encoding is modified not because RoPE is “bad,” but because its usual placement blocks an inference optimization the architecture wants to exploit.

23. MLA实际缓存什么

对应原文第 25 页：点击查看相邻原文对照页

23. MLA 实际缓存的内容 对于 解耦 RoPE（Decoupled RoPE），DeepSeek-V2 每层每个 词元（Token） 缓存两个片段：

1. 联合K/V LatentcKV

2. 共享位置密钥kR

因此 缓存有效载荷（Cache Payload） 是

标量，或

请注意相对于普通 K/V 缓存消失的内容：

- 对于单独缓存的完整K 和V 张量，主因数不为2；

- 没有描述常规键值头（KV Heads） 集合的HKV 术语。

这就是为什么将 MLA 强行纳入标准公式 2LT HKV dhb 在概念上是错误的。缓存对象不同。

对于可变长度批次，理想化的有效负载可概括为

推理服务（Serving） 分配器和 内核（Kernels） 在此架构有效负载之上添加了自己的实现细节。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

23. What MLA actually caches

With decoupled RoPE, DeepSeek-V2 caches two pieces per token per layer:

1. the joint K/V latent $\boldsymbol{c}_t^{KV} \in \mathbb{R}^{d_c}$;
2. the shared positional key $\boldsymbol{k}_t^R \in \mathbb{R}^{d_h^R}$.

Therefore the cache payload is

$$N_{\text{MLA}} = LT(d_c + d_h^R)$$

scalars, or

$$\boxed{M_{\text{MLA}} = LT(d_c + d_h^R)b.}$$

Notice what disappeared relative to ordinary K/V caching:

- no leading factor of 2 for separately cached full K and V tensors;
- no H_{KV} term describing a collection of conventional KV heads.

That is why forcing MLA into the standard formula $2LTH_{KV}d_hb$ is conceptually wrong. The cache object is different.

For variable-length batches, the idealized payload generalizes to

$$M_{\text{MLA}} = L(d_c + d_h^R)b\sum_r T_r.$$

Serving allocators and kernels add their own implementation details on top of this architectural payload.

24. DeepSeek-V2的具体MLA尺寸

对应原文第 26 页；点击查看相邻原文对照页

24. DeepSeek-V2的具体MLA维度 DeepSeek-V2使用60个Transformer 模型（Transformer）层，隐藏维度 5120、128 注意力头（Attention Heads），内容头宽度（Head Width） dh = 128，KV潜在宽度dc = 512，查询（Query）潜在宽度1536，以及解耦 RoPE（Decoupled RoPE） 宽度 dR

重要的比率是

因此，每层每个 词元（Token） 的缓存有效负载为

h = 4.5dh = 576 标量。

跨越 60 层：

每个 词元（Token） 的标量。这与论文 注意力机制（Attention） 消融表中报告的大型 MLA 模型（Model） 的 34.6K 元素数字相匹配。 [4]

这里出现了一个微妙但有价值的架构课程：DeepSeek-V2’sdh = 128 和 128 头（Heads） 并不意味着 16,384 维的 残差流（Residual Stream）。 模型（Model） 隐藏尺寸为 5120。头宽度（Head Width） 和隐藏宽度是架构参数，而不是每个设计中必须满足 dmodel = Hd h 的恒等式。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

24. DeepSeek-V2's concrete MLA dimensions

DeepSeek-V2 uses 60 Transformer layers, hidden dimension 5120, 128 attention heads, content head width $d_h = 128$, KV latent width $d_c = 512$, query latent width 1536, and decoupled RoPE width $d_h^R = 64$.[4]

The important ratios are

$$d_c = 4d_h, \qquad d_h^R = \frac{d_h}{2}.$$

Therefore the cached payload per token per layer is

$$d_c + d_h^R = 4.5d_h = 576 \text{ scalars.}$$

Across 60 layers:

$$60 \times 576 = 34{,}560$$

scalars per token. That matches the 34.6K-element figure reported for the large MLA model in the paper's attention ablation table.[4]

A subtle but valuable architecture lesson appears here: DeepSeek-V2's $d_h = 128$ and 128 heads do not imply a 16,384-dimensional residual stream. The model hidden dimension is 5120. Head width and hidden width are architectural parameters, not identities that must satisfy $d_{\text{model}} = Hd_h$ in every design.

25. 为什么论文说 MLA 就像“有 2.25 个组的 GQA”

对应原文第 27 页；点击查看相邻原文对照页

25. 为什么论文说 MLA 就像“具有 2.25 个组的 GQA”对于普通 GQA，per-词元（Token） 每层缓存包含 2Gdh 标量，其中 G 是 KV 组数/头（Heads）。对于 DeepSeek-V2 MLA，它包含

等效有效负载大小：

这是缓存大小的等效项，而不是 2.25 键值头（KV Heads） 的字面架构。人数是离散的。比较表明 DeepSeek-V2 的 MLA 缓存存储的元素与假设的具有 2.25 个组的传统 GQA 缓存一样多。 [4]

这种比较很有用，因为它说明了为什么 MLA 不仅仅是另一个头共享点。它的缓存可以占用不对应于任何整数常规 KV-head 计数的大小，因为它存储的对象是潜在的加上位置组件。

不要将“相当于具有 2.25 个组的 GQA”转换为“MLA 有 2.25 键值头（KV Heads）”。事实并非如此。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

25. Why the paper says MLA is like “GQA with 2.25 groups”

For ordinary GQA, the per-token per-layer cache contains

$$2Gd_h$$

scalars, where G is the number of KV groups/heads. For DeepSeek-V2 MLA, it contains

$$d_c + d_h^R = 4.5d_h.$$

Equating payload sizes:

$$2Gd_h = 4.5d_h \quad \Rightarrow \quad G = 2.25.$$

This is a **cache-size equivalence**, not a literal architecture with 2.25 KV heads. Head counts are discrete. The comparison says that DeepSeek-V2’s MLA cache stores as many elements as a hypothetical conventional GQA cache with 2.25 groups.[4]

The comparison is useful precisely because it shows why MLA is not simply another head-sharing point. Its cache can occupy a size that does not correspond to any integer conventional KV-head count, because its stored object is a latent plus a positional component.

> Do not turn “equivalent to GQA with 2.25 groups” into “MLA has 2.25 KV heads.” It does not.

26. 示例性 MHA/GQA/MQA/MLA 缓存比较

对应原文第 28 页：点击查看相邻原文对照页

回到前面用于说明的设置：

以 MLA 规模的示例为例，采用 DeepSeek-V2 的维度 dc = 512、dR_h = 64，可得到以下比较。

MHA（32 个 KV 头）：缓存为 4 GiB，相对 MHA 为 1 倍；保存 32 个 K 头与 32 个 V 头。

GQA（8 个 KV 头）：缓存为 1 GiB，相对 MHA 为 1/4；保存 8 个 K 头与 8 个 V 头。

MQA（1 个 KV 头）：缓存为 128 MiB，相对 MHA 为 1/32；保存 1 个 K 头与 1 个 V 头。

MLA 规模的示例：缓存为 288 MiB，相对 MHA 为 0.0703 倍；保存 512 维潜在表示与 64 维 RoPE 键（Key）。

一个值得注意的结果是：仅比较缓存容量（Cache Capacity），MQA 比这里的 MLA 参数化方案更小。

MLA 的目的不是“始终产生最小的缓存”，而是在大幅压缩缓存的同时，提供更丰富的、学习得到的注意力机制（Attention）。

这一理解框架（Mental Model）比仅按内存大小给这四种方法排序更准确。

26. MHA/GQA/MQA/MLA 缓存的说明性比较

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

26. An illustrative MHA/GQA/MQA/MLA cache comparison

Return to the earlier educational setup:

$$L = 32, \quad T = 8192, \quad H_Q = 32, \quad d_h = 128, \quad b = 2.$$

For an MLA-sized example, use DeepSeek-V2's ratios $d_c = 512$ and $d_h^R = 64$. Then

$$M_{\text{MLA}} = 32 \cdot 8192 \cdot 576 \cdot 2 = 301{,}989{,}888 \text{ bytes} = 288 \text{ MiB}.$$

Variant	Cache	vs. MHA	What is cached
MHA, 32KV	4 GiB	1x	32 K + 32 V heads
GQA, 8KV	1 GiB	1/4	8 K + 8 V heads
MQA, 1KV	128 MiB	1/32	1 K + 1 V head
MLA-sized example	288 MiB	0.0703x	512-d latent + 64-d RoPE key

A useful surprise is that MQA is smaller than this MLA parameterization in raw cache capacity. MLA's purpose is not "always produce the smallest cache." It aims for a richer learned attention mechanism while still compressing the cache aggressively.

That is a much more accurate mental model than ranking the four methods by memory alone.

27. 93.3% DeepSeek-V2 数字，范围正确

对应原文第 29 页；点击查看相邻原文对照页

27. 93.3% 的 DeepSeek-V2 数字，范围正确 DeepSeek-V2 的摘要报告称，与 DeepSeek 67B（其早期的 模型（Model））相比，键值缓存（KV Cache） 减少了 93.3%。 [4]

这个数字经常被重复，就好像“MLA 将 键值缓存（KV Cache） 减少了 93.3%”。这种概括是不合理的。MLA 缓存减少取决于 基线（Baseline） 架构和 值（Values），例如 HKV、dh、dc、dR h、层数和存储 精度（Precision）。

人们至少可以进行三种不同的比较：

- DeepSeek-V2 与本文中部署的 DeepSeek 67B 对比；

- MLA 与假设的同形状 MHA 架构；

- MLA 与具有选定数量的键值头（KV Heads） 的GQA 或MQA 架构。

他们产生不同的百分比。论文中93.3%的数字仅属于第一个比较。

出版规则。切勿在未命名 基线（Baseline） 的情况下引用压缩百分比。

“93.3%”是 DeepSeek-V2-vs-DeepSeek-67B 的结果，不是 MLA 的常数。

27. The 93.3% DeepSeek-V2 number, correctly scoped

DeepSeek-V2's abstract reports a 93.3% reduction in KV cache compared with **DeepSeek 67B**, its earlier model.[4]

That number is frequently repeated as if "MLA reduces KV cache by 93.3%." The generalization is not justified. MLA cache reduction depends on the baseline architecture and on values such as H_{KV}, d_h, d_c, d_h^R, layer count, and storage precision.

There are at least three different comparisons one might make:

- DeepSeek-V2 vs. DeepSeek 67B as deployed in the paper;
- MLA vs. a hypothetical same-shape MHA architecture;
- MLA vs. a GQA or MQA architecture with a chosen number of KV heads.

They produce different percentages. The paper's 93.3% figure belongs only to the first comparison.

> **Publication rule.** Never quote a compression percentage without naming the baseline. "93.3%" is a DeepSeek-V2-vs-DeepSeek-67B result, not a constant of MLA.

28. DeepSeek 的消融作用做什么和不做什么

对应原文第 30 页：点击查看相邻原文对照页

DeepSeek-V2 包括 MQA、8 组 GQA 和 MHA 的受控 7B 密集比较。

在四个报告的硬基准测试中，MHA 得分最高，GQA-8 中等，MQA 最低。[4]

一小部分内容说明了这种模式：

基准测试（Benchmark） MQA GQA-8 MHA BBH (3-shot EM) 33.2 35.6 37.0 MMLU (5-shot acc.) 37.9 41.2 45.2 同一篇论文分别比较了匹配的小型和大型 MoE 设置中的 MLA 和 MHA。

MLA 使用小得多的报告 KV 缓存，并且在大多数列出的 基准测试（Benchmark） 值（Values） 上更高；小 MLA 模型（Model） 在 C-Eval 上略低，而在 BBH、MMLU 和 CMMLU 上较高，而大 MLA 模型（Model） 在所有四个列出的基准上都较高。 [4]

仔细的解释是经验性的：对于这些架构、数据、训练（Training） 配方和评估，DeepSeek 发现 MLA 比比较的替代方案具有更强的缓存质量权衡。 MLA 普遍优于 MHA 或 GQA 并不是一个定理。

28. DeepSeek 的消融作用做什么和不做什么

28. What DeepSeek's ablations do and do not establish

DeepSeek-V2 includes a controlled 7B dense comparison of MQA, GQA with 8 groups, and MHA. On the four reported hard benchmarks, MHA scored highest, GQA-8 was intermediate, and MQA was lowest.[4]

A small selection illustrates the pattern:

Benchmark	MQA	GQA-8	MHA
BBH (3-shot EM)	33.2	35.6	37.0
MMLU (5-shot acc.)	37.9	41.2	45.2

The same paper separately compares MLA and MHA in matched small and large MoE setups. MLA uses far smaller reported KV caches and is higher on most listed benchmark values; the small MLA model is slightly lower on C-Eval while higher on BBH, MMLU, and CMMLU, and the large MLA model is higher on all four listed benchmarks.[4]

The careful interpretation is empirical: **for those architectures, data, training recipes, and evaluations**, DeepSeek found MLA to be a stronger cache-quality trade-off than the compared alternatives. It is not a theorem that MLA universally outperforms MHA or GQA.

29. 四种并排的设计

对应原文第 31 页；点击查看相邻原文对照页

29. 四种并行设计 MHA GQA MQA MLA 查询头（Query Heads） 许多许多许多许多 常规 键值头（KV Heads） 每个 Q 头缓存一个 少于 Q 头（Heads） 一个共享对潜在是主要缓存对象 主缓存杆 无 KV 头共享 一个共享 KV 对 低阶联合 KV 潜在 Per-词元（Token）/层缓存 2HQdh 2HKV dh 2dh 直流 + dR

持久位置块 普通 K 包含它 普通 K 包含它 普通 K 包含它 在 DeepSeek-V2 中共享 RoPE 键（Key） 主要动机 每个 查询头（Query Head） 的独立 K/V 质量/效率 中间地带 积极的 KV 共享具有更丰富容量的压缩缓存 该表强调了两个独立的想法：

头共享控制存在多少个常规 K/V 子空间。

潜在压缩控制 表示（Representation） 的存储内容。

未来的 注意力机制（Attention） 机制可以混合或扩展这些轴，因此最持久的技能是读取实际的 张量（Tensor） 形状并缓存内容，而不是记住四个首字母缩略词。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

29. The four designs side by side

	MHA	GQA	MQA	MLA
Query heads	many	many	many	many
Conventional KV heads cached	one per Q head	fewer than Q heads	one shared pair	latent is the main cache object
Primary cache lever	none	KV-head sharing	one shared KV pair	low-rank joint KV latent
Per-token/layer cache	$2H_Q d_h$	$2H_{KV} d_h$	$2d_h$	$d_c + d_h^R$
Persistent positional piece	ordinary K includes it	ordinary K includes it	ordinary K includes it	shared RoPE key in DeepSeek-V2
Main motivation	independent K/V per query head	quality/efficiency middle ground	aggressive KV sharing	compressed cache with richer capacity

The table highlights two independent ideas:

Head sharing controls how many conventional K/V subspaces exist.

Latent compression controls what representation is stored at all.

Future attention mechanisms can mix or extend these axes, so the most durable skill is to read the actual tensor shapes and cache contents rather than memorize four acronyms.

30. FlashAttention 为独立执行轴

对应原文第 32 页：点击查看相邻原文对照页

30. FlashAttention 是一个单独的执行轴 FlashAttention 更改了 稠密注意力（Dense Attention） 的执行计划，以减少 注意力机制（Attention） 中间体昂贵的 内存流量（Memory Traffic），同时计算精确密集的 Softmax 归一化（Softmax） 注意力机制（Attention）。它没有定义一个 模型（Model） 有多少个 键值头（KV Heads）。[11]

MHA/MQA/GQA/MLA 是关于 Q/K/V 表示（Representation） 的架构选择，特别是在这里，生成和缓存什么 K/V 状态。因此，模型（Model） 可以将 合并（Combine） GQA 与 FlashAttention 系列 内核（Kernel） 一起使用。当前的 PyTorch 文档明确支持多个 scaled-dot-product-注意力机制（Attention） 后端中的 GQA，包括记录约束下 CUDA 上的 FlashAttention。 [6]

这两个问题是不同的：

- 注意力机制（Attention） 架构：存在哪些Q/K/V 张量？

- 注意力机制（Attention） 内核（Kernel）：硬件上如何安排数学运算？

这与“FlashAttention 减少 键值缓存（KV Cache）”具有误导性的原因相同。 FlashAttention 主要避免实现二次 注意力机制（Attention） 中间体； MQA/GQA/MLA 攻击持久性 键（Key）/值（Value） 表示（Representation）。

30. FlashAttention is a separate execution axis

FlashAttention changes the execution schedule of dense attention to reduce expensive memory traffic for attention intermediates while computing exact dense softmax attention. It does not define how many KV heads a model has.[11]

MHA/MQA/GQA/MLA are architectural choices about Q/K/V representation and, especially here, what K/V state is produced and cached. A model can therefore combine GQA with a FlashAttention-family kernel. Current PyTorch documentation explicitly supports GQA in several scaled-dot-product-attention backends, including FlashAttention on CUDA under documented constraints.[6]

The two questions are different:

- **Attention architecture:** what Q/K/V tensors exist?
- **Attention kernel:** how is the mathematical operation scheduled on hardware?

This is the same reason "FlashAttention reduces the KV cache" is misleading. FlashAttention primarily avoids materializing quadratic attention intermediates; MQA/GQA/MLA attack the persistent key/value representation.

31. PagedAttention 是另一个不同的轴

对应原文第 33 页：点击查看相邻原文对照页

31. PagedAttention 是另一个不同的轴 PagedAttention 被引入来解决 LLM 推理服务（Serving） 中 键值缓存（KV Cache） 的内存管理：

碎片化（Fragmentation），缓存块的分配、共享和灵活放置。[12]

它没有规定 模型（Model） 是否使用 MHA、GQA 或 MQA。这些选择决定了逻辑 KV 有效负载。分页决定了如何在 推理服务（Serving） 内存系统中管理有效负载。

MLA 使区别更加清晰，因为它的持久缓存对象不是普通的完整 K/V。 推理服务（Serving） 引擎仍然需要 MLA 状态的块分配、调度和内存管理，但所管理的 表示（Representation） 是特定于体系结构的。

因此，有用的三轴 模型（Model） 是：

1. 表示（Representation）：MHA/GQA/MQA/MLA。

2. 内核（Kernel）执行：标准注意力机制（Attention） / FlashAttention系列内核（Kernels） /其他优化的内核（Kernels）。

3. 推理服务（Serving）内存管理：连续分配/分页缓存/卸载/量化（Quantization）/前缀重用。

优化一个轴并不会让其他轴消失。

31. PagedAttention is another different axis

PagedAttention was introduced to address memory management of the KV cache in LLM serving: fragmentation, allocation, sharing, and flexible placement of cache blocks.[12]

It does not prescribe whether a model uses MHA, GQA, or MQA. Those choices determine the logical KV payload. Paging determines how that payload is managed in a serving memory system.

MLA makes the distinction even clearer because its persistent cache object is not ordinary full K/V. A serving engine can still need block allocation, scheduling, and memory management for MLA state, but the representation being managed is architecture-specific.

A useful three-axis model is therefore:

1. **Representation:** MHA / GQA / MQA / MLA.
2. **Kernel execution:** standard attention / FlashAttention-family kernels / other optimized kernels.
3. **Serving memory management:** contiguous allocation / paged cache / offload / quantization / prefix reuse.

Optimizing one axis does not make the others disappear.

32. 当前 PyTorch GQA 行为

对应原文第 34 页；点击查看相邻原文对照页

32. 当前 PyTorch GQA 行为 截至 2026 年 9 月，torch.nn.function.scaled_dot_product_注意力机制（Attention） 公开了一个 enable_gqa 选项，并将 GQA 记录为实验性功能。 [6]

记录的形状符号区分 Hq（查询头（Query Heads） 的数量）和 H（键值头（Key/Value Heads） 的数量）。当前的限制包括

该文档包括一个参考等效的 Python 实现，该实现使用 Repeat_interleave 扩展了 键值头（Key/Value Heads）。这样做是为了匹配 查询（Query） 头计数。该片段描述了等效的语义；它不应该被理解为声称每个优化的后端物理上都实现了重复的 K/V 张量。

当前文档还指出了后端特定的支持和限制，这些支持和限制的变化速度比架构数学更快。因此，生产代码应将已安装的框架和 内核（Kernel） 文档视为设备、dtype、掩码（Mask）、嵌套 张量（Tensor） 和 GQA 支持的权威。

框架支持是特定于版本的。 GQA的数学定义是稳定的； 融合内核（Fused Kernel） 接受给定的 张量形状（Tensor Shape） 则不接受。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

32. Current PyTorch GQA behavior

As of September 2026, `torch.nn.functional.scaled_dot_product_attention` exposes an `enable_gqa` option and documents GQA as an experimental feature.[6]

The documented shape notation distinguishes H_q, the number of query heads, from H, the number of key/value heads. Current constraints include

$$H_q \bmod H = 0, \qquad H_K = H_V.$$

The documentation includes a reference-equivalent Python implementation that expands key/value heads with `repeat_interleave`. It does so to match query-head count. The snippet describes equivalent semantics; it should not be read as a claim that every optimized backend physically materializes repeated K/V tensors.

Current documentation also notes backend-specific support and limitations, which can change faster than the architectural mathematics. Therefore production code should treat installed framework and kernel documentation as authoritative for device, dtype, mask, nested-tensor, and GQA support.

> Framework support is version-specific. The mathematical definition of GQA is stable; which fused kernel accepts a given tensor shape is not.

33. 如何读取 模型（Model） 配置

对应原文第 35 页；点击查看相邻原文对照页

33. 如何读取 模型（Model） 配置 许多 Transformer 模型（Transformer） 配置公开两个单独的字段：

n u m _ key _ value _ head s = Hkv Current Hugging Face Llama 文档规定了通常的解释：[7]

- HKv == Hq ⇒ MHA；

- HKv == 1 ⇒ MQA；

- 1 < HKv < Hq⇒GQA。

假设 解码器（Decoder） 有 48 层，其中 64 个 查询头（Query Heads）、8 个 键值头（KV Heads）、128 个 头宽度（Head Width） 和 BF16 缓存元素。对于 16,384-词元（Token） 单序列，

该架构是 GQA，每个 KV 头有 8 个 查询头（Query Heads）。

不要盲目地将这个配置读取规则应用于MLA。 MLA 需要特定于体系结构的字段，例如 KV 潜在等级和解耦 RoPE（Decoupled RoPE） 维度，因为 conventionnum_键（Key）_值（Value）_头（Heads） 不再描述缓存。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

33. How to read a model configuration

Many Transformer configs expose two separate fields:

```
num_attention_heads = Hq
num_key_value_heads = Hkv
```

Current Hugging Face Llama documentation states the usual interpretation:[7]

- `Hkv == Hq` $\Rightarrow$ MHA;
- `Hkv == 1` $\Rightarrow$ MQA;
- $1 <$ `Hkv` $<$ `Hq` $\Rightarrow$ GQA.

Suppose a decoder has 48 layers, 64 query heads, 8 KV heads, head width 128, and BF16 cache elements. For a 16,384-token single sequence,

$$M_{\mathrm{KV}} = 2 \cdot 48 \cdot 16384 \cdot 8 \cdot 128 \cdot 2 = 3 \text{ GiB}.$$

The architecture is GQA with eight query heads per KV head.

Do not apply this config-reading rule blindly to MLA. MLA needs architecture-specific fields such as KV latent rank and decoupled RoPE dimension because conventional `num_key_value_heads` no longer describes the cache.

34. 常见错误

对应原文第 36 页；点击查看相邻原文对照页

34. 常见错误“MQA 有一个 查询头（Query Head）。”错误。 MQA 在多个 查询头（Query Heads） 之间共享一对 K/V 头。

“GQA 减少了 注意力头（Attention Heads） 的数量。”通常会产生误导。它减少了 键值头数量（KV-head Multiplicity），同时保留了 查询（Query） 头数。

“GQA 使 稠密注意力（Dense Attention） 算术线性。”错误。改变K/V 表示（Representation），可以降低投影/缓存成本；它不会消除密集的 词元（Token） 相互作用。

“MLA 只是更激进的 GQA。”错误。 MLA 的定义缓存是 DeepSeek 设计中的低阶联合 K/V 潜在缓存加上一个小型位置 键（Key）。

“MLA 始终使用比 MQA 更少的缓存。”错误。使用 DeepSeek-V2 的 4.5dh 缓存有效载荷（Cache Payload），等宽 MQA 每层存储 2dh/词元（Token）。 MLA 正在追求不同的质量与能力的权衡。

“DeepSeek 的 93.3% 是 MLA 压缩率。”错误。这是报告的 DeepSeek-V2 与 DeepSeek 67B 的比较。

“FlashAttention、GQA 和 PagedAttention 是同一优化的竞争版本。”错误的。它们在不同的轴上运行：执行计划、表示（Representation） 和缓存管理。

“头维度（Head Dimension） 必须等于 模型（Model） 宽度除以人数。”常见，但不通用。

34. Common mistakes

“MQA has one query head.” False. MQA has one K/V head pair shared across multiple query heads.

“GQA reduces the number of attention heads.” Usually misleading. It reduces KV-head multiplicity while retaining the query-head count.

“GQA makes dense attention arithmetic linear.” False. It changes K/V representation and can reduce projection/cache costs; it does not remove dense token interactions.

“MLA is just more aggressive GQA.” False. MLA’s defining cache is a low-rank joint K/V latent plus a small positional key in the DeepSeek design.

“MLA always uses less cache than MQA.” False. With DeepSeek-V2’s $4.5d_h$ cache payload, equal-width MQA stores $2d_h$ per layer/token. MLA is pursuing a different quality-capacity trade-off.

“DeepSeek’s 93.3% is the MLA compression ratio.” False. It is the reported DeepSeek-V2 vs. DeepSeek 67B comparison.

“FlashAttention, GQA, and PagedAttention are competing versions of the same optimization.” False. They operate on different axes: execution schedule, representation, and cache management.

“Head dimension must equal model width divided by head count.” Common, but not universal.

35. 实用的决策框架

对应原文第 37 页；点击查看相邻原文对照页

35. 实用的决策框架 在阅读或设计 注意力机制（Attention） 架构时，请按顺序提出以下问题：

1. 存在多少个 查询头（Query Heads）？ 这告诉您产生了多少个不同的 查询（Query） 侧 注意力机制（Attention） 通道。

2. 存在多少个常规 键值头（KV Heads）？ 如果这低于 查询（Query） 头数，则 模型（Model） 使用 MQA/GQA 系列中的 KV 共享。

3. 解码过程中到底保留了什么？ 完整的 K/V 张量、潜在张量、量化后的（Quantized） 状态、选定的 词元（Tokens） 还是其他？

4. 位置如何表示？ RoPE 或其他位置机制可以影响投影重新安排的合法性。

5. 哪些成本实际上下降了？ 投影 浮点运算次数（FLOPs）、缓存容量（Cache Capacity）、缓存流量、注意力机制（Attention） 算术、分配器开销（Allocator Overhead） 或实际运行 延迟（Latency） 是不同的指标。

6. 证据（Evidence） 支持什么质量声明？ 训练（Training） 设置和 基准测试（Benchmark） 很重要。

这个框架比记住从 MHA 到 MQA 到 GQA 到 MLA 的线性演变更持久。架构不会沿着一个轴发展。他们以不同的组合交换 表示（Representation） 容量、内存占用（Memory Footprint）、带宽、算术和实现复杂性。

紧凑型 理解框架（Mental Model）:MHA 为每个 查询头（Query Head） 提供了自己的 K/V 对。 MQA 在所有 查询（Queries） 中共享一对。 GQA 在每个 查询（Query） 组中共享一对。 MLA 将存储的对象从完整的每头 K/V 更改为学习的 潜在表示（Latent Representation） 加上一个小的位置组件。

35. A practical decision framework

When reading or designing an attention architecture, ask these questions in order:

1. **How many query heads exist?** This tells you how many distinct query-side attention channels are produced.
2. **How many conventional KV heads exist?** If this is lower than query-head count, the model uses KV sharing in the MQA/GQA family.
3. **What exactly is persisted during decode?** Full K/V tensors, a latent, a quantized state, selected tokens, or something else?
4. **How is position represented?** RoPE or another positional mechanism can affect what projection rearrangements are legal.
5. **Which costs actually fall?** Projection FLOPs, cache capacity, cache traffic, attention arithmetic, allocator overhead, or wall-clock latency are different metrics.
6. **What evidence supports the quality claim?** Training setup and benchmark matter.

This framework is more durable than memorizing a linear evolution from MHA to MQA to GQA to MLA. Architectures do not evolve on one axis. They trade representation capacity, memory footprint, bandwidth, arithmetic, and implementation complexity in different combinations.

> **The compact mental model:** MHA gives every query head its own K/V pair. MQA shares one pair across all queries. GQA shares one pair within each query group. MLA changes the stored object from full per-head K/V to a learned latent representation plus a small positional component.

36. 与理解人工智能系列其余部分的联系

对应原文第 38 页；点击查看相邻原文对照页

36. 与理解人工智能系列其余部分的联系本主题位于之前四本手册的交叉点上：

注意力机制（Attention）。 MHA是基线（Baseline）多头机构； MQA/GQA/MLA 保留 稠密注意力（Dense Attention）， 同时更改 查询（Query）/键（Key）/值（Value） 表示的组织方式。

键值缓存（KV Cache）。缓存公式使 键值头数量（KV-head Multiplicity） 的系统后果变得明确。

MLA 然后演示了为什么普通缓存公式是特定于体系结构的而不是通用的。

绳索。 MLA 的 解耦 RoPE（Decoupled RoPE） 存在是因为位置相关的旋转会干扰用于潜在 K/V 缓存的投影吸收（Projection Absorption）。

FlashAttention。 FlashAttention 优化了 注意力机制（Attention） 的运行方式； MQA/GQA/MLA 改变 注意力机制（Attention） 张量的存在以及保留的状态。

这些概念共同构成了一个有用的层次结构：

数学注意力机制（Attention）→头架构→位置编码（Positional Encoding）→持久缓存→内核（Kernel）执行。

将这些层分开是准确推理现代 Transformer 模型（Transformer） 推理（Inference） 的最佳方法之一。

36. Connections to the rest of the Understanding AI series

This topic sits at the intersection of four previous handbooks:

Attention. MHA is the baseline multi-head mechanism; MQA/GQA/MLA preserve dense attention while changing how query/key/value representations are organized.

KV Cache. The cache formula makes the systems consequences of KV-head multiplicity explicit. MLA then demonstrates why the ordinary cache formula is architecture-specific rather than universal.

RoPE. MLA's decoupled RoPE exists because position-dependent rotations interfere with the projection absorption used for latent K/V caching.

FlashAttention. FlashAttention optimizes how attention runs; MQA/GQA/MLA change what attention tensors exist and what state is retained.

Together, these concepts form a useful hierarchy:

$$\begin{aligned}&\text{mathematical attention} \to \text{head architecture} \to \text{positional encoding}\\ &\to \text{persistent cache} \to \text{kernel execution.}\end{aligned}$$

Keeping those layers separate is one of the best ways to reason accurately about modern Transformer inference.

参考文献和进一步阅读

对应原文第 39 页；点击查看相邻原文对照页

参考文献和进一步阅读

[1] Vaswani, A. 等人。注意力机制（Attention） 就是您所需要的。 NeurIPS 2017。
https://arxiv.org/abs/1706.03762

[2] Shazeer，N.Fast Transformer 模型（Transformer） 解码：您只需要一个写头。 2019。
https://arxiv.org/abs/1911.02150

[3] Ainslie, J. 等人。 GQA：训练（Training） 来自多头检查点的通用多 查询（Query） Transformer 模型（Transformer） 模型（Models）。 EMNLP 2023。https://aclanthology.org/2023.emnlp-main.298/

[4] DeepSeek-AI 等人。DeepSeek-V2：一种强大、经济、高效的 混合专家（Mixture-of-Experts） 语言 模型（Model）。 2024。https://arxiv.org/abs/2405.04434

[5] Su, J. et al.RoFormer：使用 旋转位置嵌入（Rotary Position Embedding） 增强 Transformer 模型（Transformer）。神经计算 2024；

预印本于 2021 年首次发布。https://arxiv.org/abs/2104.09864

[6] 火炬。 torch.nn.function.scaled_dot_product_attention文档。当前行为于 2026 年 9 月检查。
https://docs.pytorch.org/docs/stable/ generated/torch.nn.function.scaled_ dot_product_注意力机制（Attention）

[7] Hugging Face Transformers.Llama 配置文档，包括 num_注意力机制（Attention）_头（Heads） 和 num_键（Key）_值（Value）_头（Heads）。当前文档于 2026 年 9 月检查。
https://huggingface.co/docs/transformers/main/模型（Model）_doc/llama

[8] Touvron, H. 等人。Llama 2：开放基础和微调聊天 模型（Models）。 2023。https://arxiv。

组织/abs/2307.09288

[9] Jiang, A. Q. et al.Mistral 7B。 2023。https://arxiv.org/abs/2310.06825

[10] Meta AI.Meta Llama 简介 3. 2024. https://ai.meta.com/blog/meta-llama-3/

[11] Dao, T. 等人FlashAttention：具有 IO 感知功能的快速且内存高效的 精确注意力（Exact Attention）。神经信息处理系统

2022。https://arxiv.org/abs/2205.14135

[12] Kwon, W. 等人。使用 PagedAttention 进行 大语言模型（Large Language Model） 推理服务（Serving） 的高效内存管理。

SOSP 2023。https://arxiv.org/abs/2309.06180 来源纪律。本手册中特定论文的实证结果作为引用实验的结果呈现，而不是普遍保证。框架行为具有明确的时间范围，可以更改。

References and further reading

[**1**] Vaswani, A. et al. *Attention Is All You Need.* NeurIPS 2017. https://arxiv.org/abs/1706.03762

[**2**] Shazeer, N. *Fast Transformer Decoding: One Write-Head Is All You Need.* 2019. https://arxiv.org/abs/1911.02150

[**3**] Ainslie, J. et al. *GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints.* EMNLP 2023. https://aclanthology.org/2023.emnlp-main.298/

[**4**] DeepSeek-AI et al. *DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model.* 2024. https://arxiv.org/abs/2405.04434

[**5**] Su, J. et al. *RoFormer: Enhanced Transformer with Rotary Position Embedding.* Neurocomputing 2024; preprint first posted 2021. https://arxiv.org/abs/2104.09864

[**6**] PyTorch. *torch.nn.functional.scaled_dot_product_attention* documentation. Current behavior checked September 2026. https://docs.pytorch.org/docs/stable/generated/torch.nn.functional.scaled_dot_product_attention

[**7**] Hugging Face Transformers. *Llama configuration documentation*, including `num_attention_heads` and `num_key_value_heads`. Current documentation checked September 2026. https://huggingface.co/docs/transformers/main/model_doc/llama

[**8**] Touvron, H. et al. *Llama 2: Open Foundation and Fine-Tuned Chat Models.* 2023. https://arxiv.org/abs/2307.09288

[**9**] Jiang, A. Q. et al. *Mistral 7B.* 2023. https://arxiv.org/abs/2310.06825

[**10**] Meta AI. *Introducing Meta Llama 3.* 2024. https://ai.meta.com/blog/meta-llama-3/

[**11**] Dao, T. et al. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness.* NeurIPS 2022. https://arxiv.org/abs/2205.14135

[**12**] Kwon, W. et al. *Efficient Memory Management for Large Language Model Serving with PagedAttention.* SOSP 2023. https://arxiv.org/abs/2309.06180

Source discipline. Paper-specific empirical results in this handbook are presented as results of the cited experiments, not universal guarantees. Framework behavior is explicitly time-scoped where it can change.

参考图：针对哪个问题阅读什么内容

对应原文第 40 页；点击查看相邻原文对照页

参考图：针对哪个问题阅读什么内容 问题 最佳起始来源 多头注意力（Multi-head Attention） 从哪里来？ 瓦斯瓦尼等人。 [1]

为什么提出MQA？ Shazeer [2] GQA是如何定义的，MHA是如何转换的

安斯利等人。 [3]

MLA 到底缓存什么？ DeepSeek-V2 [4]，第 2.1 节 为什么 MLA 解耦 RoPE？ DeepSeek-V2 [4]，第 2.1.3 节 2.25 组和 93.3% 的数字来自哪里？

DeepSeek-V2 [4] 受控 MQA/GQA/MHA 和 MLA/MHA 消融报告了什么？

DeepSeek-V2 [4]，附录 D 当前的 PyTorch 如何公开 GQA？ PyTorch SDPA 文档 [6] 当前的 Llama 配置如何编码 KV 头计数？

Hugging Face Llama 文档 [7] 哪些 Llama 2 尺寸使用了 GQA？ Llama 2 报告 [8] Mistral 7B 中哪里使用了 GQA？ Mistral 7B [9] FlashAttention 和 PagedAttention 与头架构有何不同？

FlashAttention [11]、PagedAttention [12] 实现支持的变化比数学更快。对于已部署的系统，将确切安装的框架、模型（Model） 代码和 注意力机制（Attention）-内核（Kernel） 文档视为受支持设备、dtypes、掩码、头维度（Head Dimensions）、缓存布局和融合 GQA/MLA 行为的权威。

面向人工智能工程师、开发人员和认真学习者的简短技术手册

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

Reference map: what to read for which question

Question	Best starting source
Where did multi-head attention come from?	Vaswani et al. [1]
Why was MQA proposed?	Shazeer [2]
How is GQA defined, and how was MHA converted to it?	Ainslie et al. [3]
What exactly does MLA cache?	DeepSeek-V2 [4], Section 2.1
Why does MLA decouple RoPE?	DeepSeek-V2 [4], Section 2.1.3
Where do the 2.25-group and 93.3% numbers come from?	DeepSeek-V2 [4]
What did the controlled MQA/GQA/MHA and MLA/MHA ablations report?	DeepSeek-V2 [4], Appendix D
How does current PyTorch expose GQA?	PyTorch SDPA docs [6]
How do current Llama configs encode KV-head count?	Hugging Face Llama docs [7]
Which Llama 2 sizes used GQA?	Llama 2 report [8]
Where is GQA used in Mistral 7B?	Mistral 7B [9]
How are FlashAttention and PagedAttention different from head architecture?	FlashAttention [11], PagedAttention [12]

Implementation support changes faster than the mathematics. For deployed systems, treat the exact installed framework, model code, and attention-kernel documentation as authoritative for supported devices, dtypes, masks, head dimensions, cache layouts, and fused GQA/MLA behavior.